Style-Preserving Hindi/Urdu to English Poetic Translation

A Three-Tier Comparison: Rule-Based, Seq2Seq with Bahdanau Attention, and Fine-Tuned Transformers

Adarsh Singh — Individual Final Report

DATS 6312 — Natural Language Processing
Spring 2026

Instructor: Dr. Amir Jafari

The George Washington University

April 29, 2026

GitHub Repository: https://github.com/drsh0755/Final-Project-Group2

1 Introduction

1.1 Project Overview

This project developed a multi-tier neural machine translation system targeting Hindi/Urdu classical poetry (ghazals, nazms, dohas) translated into English while preserving poetic style. Standard machine-translation tools such as Google Translate produce semantically literal output that strips away the rhythm, end-rhyme, and cultural metaphor that make ghazals what they are. The objective here was to build a pipeline that does better than literal translation on style, while still being faithful in meaning.

The system progresses through three model tiers of increasing sophistication: (1) a rule-based dictionary baseline implemented from scratch with an Urdu/Hindi–English lexicon, (2) a Sequence-to-Sequence (Seq2Seq) LSTM with Bahdanau additive attention implemented from scratch in PyTorch, and (3) the pretrained Helsinki-NLP `opus-mt-hi-en` (MarianMT) Transformer fine-tuned on poetry data. Each tier was iterated through multiple versions; eight model checkpoints were preserved on disk and exposed through a Streamlit demonstration application for the final presentation. Evaluation uses both standard MT metrics (BLEU vs. literal and style references) and two custom style metrics (Rhyme Density Score, Syllable Alignment Score).

1.2 Development Approach

Development began with the data layer because of the unusual nature of the task. The Kaggle Urdu Ghazal Dataset (32 poets across en/, hi/, ur/ folders) provides parallel transliterations but no English meaning translations, so I first built a translation-generation pipeline that produces three English reference columns per source line (`en_t` literal via deep-translator, `en_anthropic` style-preserving via Anthropic Claude on AWS Bedrock, and `en_mistral` style-preserving via Mistral Large). Once the dataset was in place, the three model tiers were implemented sequentially, each with a v1 and a v2 (and Tier 3 additionally with v3a and v3b for two further fine-tuning strategies). Each version's checkpoint was preserved so that the final evaluation script could compare all eight models side-by-side.

1.3 Algorithm Development

The data pipeline (Kaggle download, multi-API translation generation, IIT Bombay corpus preparation, JSON-to-CSV conversion with stratified split) was implemented first. The three model tiers were then implemented sequentially, each with v1 and v2 (and Tier 3 additionally with v3a and v3b for two further fine-tuning strategies). A unified evaluation script with the two custom style metrics was used to compare all eight model checkpoints side-by-side, and a Streamlit demonstration application was built on top of those checkpoints for the final presentation. AI assistants were used as a coding tool throughout; all architectural and hyperparameter decisions, debugging, and final review were done by me.

1.3.1 Bahdanau Additive Attention Equations

At decoder step t with previous decoder hidden state s_{t-1}, the unnormalized alignment score against encoder hidden state h_j^{enc} at source position j is:

$$e_{t,j} = v^\top \tanh\big(W_a\, s_{t-1} + U_a\, h_j^{\text{enc}}\big) \tag{1}$$

$$\alpha_{t,j} = \frac{\exp(e_{t,j})}{\sum_{k=1}^{n} \exp(e_{t,k})} \tag{2}$$

$$c_t = \sum_{j=1}^{n} \alpha_{t,j}\, h_j^{\text{enc}} \tag{3}$$

where W_a, U_a, v are learnable parameters, n is the source length, and c_t is the context vector consumed by the decoder.

1.3.2 LSTM Cell Equations

The encoder and decoder use stacked LSTM cells governed by:

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \tag{4}$$

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \tag{5}$$

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \tag{6}$$

$$\tilde{g}_t = \tanh(W_g x_t + U_g h_{t-1} + b_g) \tag{7}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{g}_t \tag{8}$$

$$h_t = o_t \odot \tanh(c_t) \tag{9}$$

where σ is the sigmoid function and $\odot$ denotes element-wise multiplication.

1.3.3 Scaled Dot-Product Attention (Tier 3)

The Transformer's multi-head attention uses scaled dot-product attention (Vaswani et al., 2017):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \tag{10}$$

applied in parallel across h=8 heads and concatenated.

1.3.4 Performance Metrics

BLEU (Papineni et al., 2002) is the geometric mean of modified n-gram precisions p_n tempered by a brevity penalty:

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^{4} w_n \log p_n\right), \qquad \text{BP} = \begin{cases} 1 & \text{if } c > r \\ \exp(1 - r/c) & \text{if } c \le r \end{cases} \tag{11}$$

where c is the candidate length and r is the reference length, with $w_n = 1/4$.

Rhyme Density Score (custom). Let L be the set of non-empty output lines; let $\text{last}_3(\ell)$ denote the last three characters of the final word of line ℓ (a coarse rhyme proxy). Then:

$$\text{RhymeDensity} = \frac{1}{|L|} \sum_{\ell \in L} \mathbf{1}\big[\, \big|\{\ell' \in L : \text{last}_3(\ell') = \text{last}_3(\ell)\}\big| \geq 2 \,\big] \tag{12}$$

Syllable Alignment Score (custom). Let $\sigma_{\text{src}}(\ell)$ be the Devanagari syllable count of source line ℓ (vowels and matras) and $\sigma_{\text{tgt}}(\ell)$ the English syllable count of the corresponding target line (vowel-group heuristic), with $m = \min(|L_{\text{src}}|, |L_{\text{tgt}}|)$:

$$\text{SyllAlign} = 1 - \frac{1}{m} \sum_{\ell=1}^{m} \frac{|\sigma_{\text{src}}(\ell) - \sigma_{\text{tgt}}(\ell)|}{\max\big(\sigma_{\text{src}}(\ell),\, \sigma_{\text{tgt}}(\ell)\big)} \tag{13}$$

2 Individual Work Description

2.1 Architectural Background

Hindi/Urdu poetry has a rigid prosodic skeleton: ghazals follow a strict *radif* (refrain) and *qafia* (rhyming word) pattern, line lengths are governed by quantitative meter, and the imagery (candle-and-moth, wine-and-beloved, the lover-as-supplicant) is recurrent and culturally loaded. A translator that cares only about word-by-word semantic match (Google Translate, deep-translator) produces fluent English prose with no surviving meter or rhyme; a translator that hallucinates rhyme without grounding in the source produces output that sounds poetic but loses meaning. The three-tier design here is meant to put a number on this trade-off rather than pick a side: Tier 1 maximally favors source-token fidelity and ignores fluency, Tier 3 maximally favors English fluency and tends to ignore form, Tier 2 sits in between.

The strongest tier (Tier 3) builds on the encoder-decoder Transformer of Vaswani et al. (2017) by way of MarianMT (Junczys-Dowmunt et al., 2018) and the Helsinki-NLP `opus-mt-hi-en` checkpoint, which was pretrained on the OPUS Hindi-English parallel corpora (Tiedemann, 2020). The middle tier (Tier 2) is a from-scratch PyTorch implementation of the Sutskever et al. (2014) encoder-decoder LSTM extended with the additive attention of Bahdanau et al. (2015).

2.2 Architecture and Training Configurations

Tier 2 (Seq2Seq LSTM) uses a 2-layer bidirectional encoder with 256-dim embeddings and 512-dim hidden states, projected through a tanh layer to initialize a 2-layer unidirectional decoder; Bahdanau attention sits between them. Training uses Adam (η=10^{-3}), batch size 64, dropout 0.3, gradient clipping at $\|\nabla\theta\| \leq 1$, teacher forcing with probability 0.5, and early stopping on validation loss with patience 10. Variable-length batching is handled with PyTorch `pack_padded_sequence`. The v2 checkpoint has 77,031,528 trainable parameters.

Tier 3 (Opus-MT fine-tune) is approximately 76M trainable parameters. v1 used a two-phase schedule: Phase 1 on 50,000 IIT Bombay general sentence pairs (3 epochs, η=5×10^{-5}, batch 64), Phase 2 on poetry *hi* $\rightarrow$ *en_anthropic* (15 epochs, η=2×10^{-5}, batch 32). v2 dropped Phase 1 entirely and lowered η to 5×10^{-6}, raised weight decay to 0.05, and lowered batch size to 16. v3a interleaved `en_anthropic` and `en_mistral` targets in alternating mini-batches; v3b applied

curriculum learning (Phase A on `en_anthropic`, then Phase B on `en_mistral` at η=2×10^{-6}). All Tier 3 training used Hugging Face `Seq2SeqTrainer` with mixed-precision (`fp16`) and beam search at inference (beams 4, `no_repeat_ngram_size` 2 for v2+, length penalty 0.8).

2.3 Feature Engineering and Preprocessing

Unlike the previous-semester stock-prediction project where feature engineering was the dominant work, NLP feature engineering here is mostly text normalization. The pipeline is:

- Devanagari Unicode normalization (NFC) to merge canonically equivalent code-point sequences.
- Whitespace and punctuation normalization, with line breaks preserved (line breaks are linguistically meaningful in poetry).
- Tokenization using IndicNLP for Devanagari and standard whitespace tokenization for English.
- For Tier 1 only: a romanization fallback (Devanagari→ITRANS via `indic-transliteration`) for tokens missing from the bilingual lexicon.
- Source-side syllable counting via Devanagari vowel/matra detection; target-side syllable counting via vowel-group heuristic. (An earlier version naively counted Latin vowels on the Devanagari side, returning ∼0 across the board; the bug was fixed by replacing the source-side counter.)

3 Detailed Implementation

3.1 Pipeline Overview

The end-to-end pipeline is organized as a sequence of stand-alone Python scripts, each runnable from the command line and each persisting its output to disk. Roughly: scripts 01–04 acquire and translate the source data, scripts 05–08 prepare features and the train/val/test split, scripts 09–19 train and evaluate the eight model versions, and the final script (20) launches the Streamlit demonstration. The full script-level execution order, with file names and one-line descriptions, is listed in Appendix A.

3.2 AWS Deployment

All deep-learning training was performed on an AWS EC2 `g5.xlarge` instance with one NVIDIA A10G GPU (24 GB VRAM), Ubuntu 22.04, CUDA 12, PyTorch 2.x, Transformers 4.57.6, sacreBLEU 2.6.0, and SentencePiece 0.2.1. Tier 1 ran in seconds on CPU. Tier 2 v2 trained at ∼80 s/epoch. Tier 3 trained at 8–12 minutes/epoch on the full poetry train split, with mixed-precision (`fp16`) enabled to halve memory use. Total GPU hours across all eight checkpoints were approximately 6 hours.

3.3 Multi-Reference Dataset Construction

The most distinctive piece of the pipeline is the multi-reference dataset itself. Where most NMT projects use a single ground-truth target per source, this project produces three English refer-

ences per Devanagari line — a literal one (`en_t`) and two style-preserving ones (`en_anthropic`, `en_mistral`) — and uses them in three different ways: `en_anthropic` as the primary training target, all three as augmentation in Tier 2 v2, and the `en_t` / `en_anthropic` pair as twin BLEU references at evaluation (BLEU vs. Literal and BLEU vs. Style respectively). This is what allows the project to put a number on the trade-off between literal fidelity and stylistic preservation in the final results table.

3.4 Streamlit Demonstration Application

The Streamlit app exposes 11 translators in a single UI: three external APIs (Google Translate via `deep_translator`; Anthropic Claude via Bedrock; Mistral Large via the Mistral API), Tier 1 v1 and v2, Tier 2 v1 and v2, and Tier 3 v1, v2, v3a, and v3b. The user pastes a Hindi poem and sees the 11 translations side-by-side, each scored against the `en_anthropic` and `en_t` references with BLEU, rhyme density, and syllable alignment. This was used as the live demo during the final presentation.

4 Results

4.1 V1 Results

The first iteration confirms the expected ordering: Tier 1 produces near-zero BLEU (it cannot string fluent English together at all), Tier 2 produces a small but positive signal, and Tier 3 jumps roughly an order of magnitude above Tier 2 because the Opus-MT base model is already a competent Hindi→English translator before any fine-tuning.

Table 1: Cross-tier comparison after v1 training (100 held-out test poems).

Metric	Tier 1 Rule-Based	Tier 2 Seq2Seq	Tier 3 Opus-MT
BLEU vs Literal	0.37	1.59	**8.36**
BLEU vs Style	0.28	1.56	**6.08**
Rhyme Density (tgt)	0.603	**0.734**	0.258
Syllable Alignment	0.143	0.371	**0.509**

4.2 V2 Results

The v2 round was where the largest gains came from. Tier 1 v2's expanded lexicon plus transliteration fallback raised dictionary coverage and produced a $\sim$2.5$\times$ BLEU jump (0.37→0.92 vs. literal). Tier 2 v2's multi-target augmentation (training on `en_anthropic`, `en_mistral`, `en_t` for the same source line) gave a $\sim$1.8$\times$ BLEU jump and acted as soft label smoothing. Tier 3 v2 (Phase 1 dropped, lower LR, higher weight decay, smaller batch) gave a more modest BLEU gain over v1 but was meaningfully better calibrated — the validation curves were no longer dominated by the Phase 1 prose-domain shift.

Table 2: Cross-tier comparison after v2 training (100 held-out test poems).

Metric	Tier 1 v2	Tier 2 v2	Tier 3 v2
BLEU vs Literal	0.92	2.84	**8.94**
BLEU vs Style	0.73	2.12	**6.84**
Rhyme Density (tgt)	0.616	**0.651**	0.252
Syllable Alignment	**0.469**	0.327	**0.520**

4.3 Final Cross-Version Results

Table 3 reports all eight checkpoints side-by-side. The Tier 3 v3 variants are interesting: v3a (interleaved styles) gives the highest BLEU vs. Style (7.07) of any model, slightly beating v2 (6.84) on the stylistic axis at the cost of ∼1 point of BLEU vs. Literal — which is exactly what one would hope for from an interleaving curriculum that teaches the model two different style targets simultaneously. v3b (curriculum learning) does not improve further. The headline number for the report is therefore Tier 3 v3a's 7.07 BLEU vs. Style.

Table 3: All eight model checkpoints, evaluated on the same 100 held-out test poems by `evaluate_all_tiers_final.py`.

Metric	T1 v1	T1 v2	T2 v1	T2 v2	T3 v1	T3 v2	T3 v3a	T3 v3b
BLEU vs Literal	0.35	0.92	2.19	2.84	**9.05**	8.94	7.90	7.77
BLEU vs Style	0.29	0.73	3.52	2.12	6.69	6.84	**7.07**	6.69
Rhyme Density (tgt)	0.600	0.616	**0.701**	0.651	0.270	0.252	0.311	0.284
Syllable Alignment	0.144	0.469	0.388	0.327	0.503	**0.520**	0.494	0.455

4.4 Training Dynamics and Overfitting

Overfitting was the dominant pathology and was tracked across all training runs:

Tier 2 v2 — clear overfitting after epoch 3. Train loss fell from 6.59 → 3.41 over epochs 1–11 while validation loss rose from 6.27 → 6.44 and validation perplexity climbed from 459 → 625. Early stopping with patience 10 retained the epoch-3 checkpoint (loss_{val}=6.13, PPL_{val}=458). Per-epoch loss and perplexity were logged to `Plots/tier2_v2_loss_curve.png`.

Tier 3 v1 (Phase 2) — validation BLEU went 21.36 → 17.97 between epochs 3 and 4. Early stopping at patience 3 retained the epoch-3 checkpoint. The decline was the single biggest reason Phase 1 was dropped in v2.

Tier 3 v2 — with Phase 1 dropped and η lowered to 5×10^{-6}, validation BLEU rose monotonically for 8 epochs before plateauing, and the final test-set BLEU (8.94) was only ∼1% below validation BLEU — much healthier generalization than v1.

4.5 Performance Summary

- Best BLEU vs. Literal: **Tier 3 v1 = 9.05**.
- Best BLEU vs. Style: **Tier 3 v3a = 7.07**.

- Best Rhyme Density: **Tier 2 v1 = 0.701** (note: this is partly an artifact of the LSTM producing repetitive line endings; see Discussion).
- Best Syllable Alignment: **Tier 3 v2 = 0.520**.
- Across all axes, no single model wins everywhere — the right model depends on whether the downstream user prioritizes literal fidelity, stylistic English, surface rhyme, or rhythmic length-matching.

4.6 Qualitative Example

Source (Hindi, Mirza Ghalib): *hazaṛoṃ ḳhvāhisheṃ aisī ki har ḳhvāhish pe dam nikle / bahut nikle mere armān lekin phir bhī kam nikle.*

Tier 1 v2 (rule-based): *thousands desires such that every desire on breath comes-out / many came-out my arman but still less came-out.*

Tier 2 v2 (LSTM): *a thousand desires of such kind that every desire takes a breath / many of my hopes came forth but still they were too few.*

Tier 3 v3a (Opus-MT, interleaved styles): *a thousand desires, each one worth dying for / many of my longings were fulfilled, yet many remain.*

Tier 3 produces idiomatic English close to a published translation; Tier 2 captures the structure but produces stilted phrasing; Tier 1 leaks Devanagari placeholders where the dictionary fails.

5 Summary and Conclusions

This project successfully delivered an end-to-end Hindi→English poetic translation pipeline with three model tiers and a total of eight trained model checkpoints, evaluated on a custom multi-reference parallel corpus built from the Kaggle Urdu Ghazal Dataset [3] (32 poets) and augmented with literal translations from deep-translator and style-preserving translations from Anthropic Claude and Mistral Large. The strongest configuration (Tier 3 v3a, Opus-MT fine-tuned with interleaved `en_anthropic`/`en_mistral` targets) achieves **BLEU vs. Style 7.07** and **BLEU vs. Literal 7.90**, comfortably above the Tier 2 LSTM baseline (3.52 / 2.19) and the Tier 1 dictionary baseline (0.29 / 0.35).

5.1 Key Learnings

- **Match the model to the task.** An early shortlist of HiNER-based encoder-only NER models had to be discarded because they cannot generate translations at all; the correct family for this task is encoder-decoder Seq2Seq (Opus-MT, mBART). Reading the model card carefully before trusting the name is now a habit.
- **More data is not always better data.** Phase 1 fine-tuning on 50K rows of IIT Bombay general parallel text actively hurt downstream poetry quality in Tier 3 v1. The pretrained Opus-MT was already specialized to general Hindi→English; adding more general data shifted it away from the poetic register that Phase 2 then had to recover. Tier 3 v2 (Phase 1 dropped) outperformed v1 on the stylistic axis.

- **Custom metrics need to be debugged like code.** The first version of the Syllable Alignment Score returned ∼0.0 across the board because it counted Latin vowels on the Devanagari source — a bug, not a result. Replacing the source-side counter with a Devanagari-aware vowel/matra detector raised Tier 3's syllable alignment from 0.0 to 0.520.
- **Surface rhyme is not poetic quality.** Tier 2's high rhyme density (0.701 for v1) reflects repetitive line endings rather than genuine rhyme. Standard MT metrics (BLEU, chrF) and surface style metrics measure different things; reporting both, with honest caveats, is more informative than picking one.
- **Plan around external dependencies.** The original plan included five LLM-generated reference columns (`en_anthropic`, `en_mistral`, `en_groq`, `en_gemini`, `en_meta`, `en_grok`); only the first two could be produced because the others' APIs would not authenticate cleanly within the time window or hit free-tier daily limits. Designing the dataset pipeline so that any single reference column being missing degrades gracefully (rather than breaking the whole training run) was the right call in retrospect.
- **Scrape-versus-curated trade-off.** The original data plan was to scrape Rekhta.org and Kavitakosh.org. Neither yielded structured plaintext (page markup was inconsistent across poems and transliterations were embedded in JavaScript-rendered widgets), so I dropped both and switched to the Kaggle dataset, which was already structured and license-clean.

5.2 Future Improvements

- Add a rhyme-aware auxiliary loss term for Tier 3, e.g., $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \lambda \cdot (1 - \text{RhymeDensity})$, trading a small amount of BLEU for a substantial rhyme-density gain.
- Extend to Urdu-Nastaliq script. The bottleneck is parallel data: a useful next step is paying for ∼500 professionally translated Urdu ghazal pairs.
- Add IndicTrans2 [4, 5] as a fourth tier once the missing-corpus issue is resolved.
- Add a constrained-beam-search variant that biases the decoder toward English words ending in a target rhyme set extracted from the source.
- Use the Dakshina dataset [6] for higher-quality transliteration and a closer-to-published Urdu reference set.
- Try LoRA / QLoRA fine-tuning of mBART-50 or NLLB-200; both are larger and multilingual, and a parameter-efficient adaptation would let me train several styles' worth of adapters without retraining the whole model.
- Replace the line-level evaluation with a poem-level human evaluation (1–5 scale on Fluency, Meaning, Poetic Feel) on a small sub-sample to validate that BLEU progress corresponds to perceived quality.

6 Percentage of Code from Internet/GenAI

Table 4: Code Attribution

Component	Total Lines	Internet/GenAI	Modified	Own Code
Data pipeline (steps 1–8)	1,210	1,030	130	50
Tier 1 v1 / v2	540	460	60	20
Tier 2 v1 / v2 (Seq2Seq + attn)	980	820	110	50
Tier 3 v1 / v2 / v3a / v3b	1,340	1,120	150	70
Evaluation scripts (3 versions)	720	600	80	40
Streamlit demo app	610	510	70	30
TOTAL	**5,400**	**4,540**	**600**	**260**

$$\text{Percentage of Code from Internet/AI} = \frac{4540 - 600}{4540 + 260} \times 100 = 82.08\% \tag{14}$$

The percentage of code from internet/GenAI (Claude) is 82.08%. This reflects extensive use of AI-assisted development tools, framework documentation examples (PyTorch's official Seq2Seq + attention tutorial, Hugging Face `Seq2SeqTrainer` boilerplate), and library-usage templates (`deep_translator`, `sacrebleu`, IndicNLP, the Anthropic and Mistral SDKs). The modified lines (600) represent adaptations and customizations made to fit project-specific requirements — changes to the LSTM dimensions and the encoder/decoder layer counts in Tier 2, the four-version training-loop variants of Tier 3, the Devanagari-aware syllable counting in the evaluation scripts, and the multi-translator orchestration in the Streamlit app. The own code (260 lines) includes project-specific logic such as the two custom style metrics (Rhyme Density Score, Syllable Alignment Score), the multi-reference dataset construction with poet-stratified poem-level splitting, and the cross-version evaluation harness.

References

[1] CFILT. `cfilt/iitb-english-hindi`. Hugging Face Datasets Hub. https://huggingface.co/datasets/cfilt/iitb-english-hindi

[2] CFILT. `cfilt/HiNER-original`. Hugging Face Datasets Hub. https://huggingface.co/datasets/cfilt/HiNER-original

[3] M. A. B. Siddiqui. "Urdu Ghazal Dataset — 32 Poets and Their Ghazals." Kaggle. https://www.kaggle.com/datasets/mabubakrsiddiq/urdu-ghazal-dataset-32-poets-and-their-ghazals

[4] AI4Bharat. "IndicTrans2" (training and inference code). GitHub. https://github.com/AI4Bharat/IndicTrans2

[5] AI4Bharat. `ai4bharat/indictrans2-indic-en-1B`. Hugging Face Model Hub. https://huggingface.co/ai4bharat/indictrans2-indic-en-1B

[6] Google Research. Dakshina Dataset (transliteration and Romanization for South Asian languages). GitHub. https://github.com/google-research-datasets/dakshina

[7] PyTorch. "NLP From Scratch: Translation with a Sequence to Sequence Network and Attention." Tutorial. https://pytorch.org/tutorials/intermediate/seq2seq_translation_tutorial.html

[8] A. Kunchukuttan. "IndicNLP Library." GitHub. https://github.com/anoopkunchukuttan/indic_nlp_library

[9] Streamlit Documentation. https://docs.streamlit.io/

A Pipeline: Script-Level Execution Order

The full project implementation consists of Python scripts organized in a sequential pipeline. The execution order on a fresh machine is given below, with one-line descriptions per script. Steps 1–8 build the dataset; steps 9–19 train and evaluate the model versions; step 20 launches the demo.

A.1 Data Acquisition Scripts

Step 1 — Kaggle dataset download. The Urdu Ghazal Dataset (32 poets, with parallel `en/`, `hi/`, `ur/` folders per poet) [3] is fetched with the Kaggle CLI:

```
kaggle datasets download mabubakrsiddiq/urdu-ghazal-dataset-32-poets-and-their-ghazals
```

Step 2 — `translate_poems_google.py`. Generates the `en_t` (literal) reference column by wrapping `deep_translator`'s `GoogleTranslator` backend. Used as the literal-quality reference in BLEU vs. Literal.

```
python translate_poems_google.py --test
python translate_poems_google.py --all
```

Step 3 — `translate_poems_anthropic.py`. Generates the `en_anthropic` (style-preserving) reference column by calling Claude Sonnet 4.5 on AWS Bedrock with a poetry-preserving system prompt that instructs the model to maintain ghazal radif/qafia structure, line count, and cultural metaphors and to return a strict JSON array. This becomes the primary training target.

```
python translate_poems_anthropic.py --test
python translate_poems_anthropic.py --all
```

Step 4 — `translate_poems_all.py`. Originally intended to add five further reference columns (`en_groq`, `en_gemini`, `en_meta`, `en_grok`, `en_mistral`). In practice, only `en_mistral` could be produced — the other four endpoints failed authentication or hit free-tier daily limits within the time window.

```
python3 translate_poems_all.py --test --only en_mistral
python3 translate_poems_all.py --all  --only en_mistral
```

Step 5 — `test_indictrans2_ghazals.py` (skipped). Scaffolded to call AI4Bharat IndicTrans2 [4, 5] for an additional reference column, but the model could not produce a usable translated corpus on the available poetry within the time budget, so this path was abandoned.

A.2 Data Preparation Scripts

Step 6 — Sync translations. The JSON outputs from Steps 2–4 in `Results/outputs/` are copied into `Data/processed/` so the downstream tier scripts share a canonical input location.

```
cp Results/outputs/*.json Data/processed/
```

Step 7 — `iitb_data_prep.py`. Downloads `cfilt/iitb-english-hindi` [1] from the Hugging Face hub, filters to a usable subset, and writes a CSV used for Tier 3 v1 Phase 1 fine-tuning.

```
python iitb_data_prep.py
```

Step 8 — `poetry_json_to_csv.py`. Flattens the per-poem JSON files into a long-format line-level table with columns `hi, en, en_t, en_anthropic, en_mistral, poet, poem_slug`. Splitting is stratified by poet at the poem level (no two lines from the same poem land in different splits); the resulting splits are 14,591 train / 2,898 val / 3,380 test line pairs.

```
python poetry_json_to_csv.py
```

A.3 Tier 1 Scripts

Step 9 — `tier1_rule_based_baseline.py`. Tier 1 v1: dictionary-based rule baseline with a 3K-entry Hindi/Urdu–English lexicon, no transliteration fallback. Lower-bound baseline.

```
python3 tier1_rule_based_baseline.py
```

Step 13 — `tier1_rule_based_baseline_v2.py`. Tier 1 v2: expanded ∼8.5K-entry lexicon plus ITRANS romanization fallback for OOV tokens. Dictionary coverage rose from 40.2% to roughly 62%.

```
python3 tier1_rule_based_baseline_v2.py
```

A.4 Tier 2 Scripts

Step 10 — `tier2_seq2seq_lstm.py`. Tier 2 v1: from-scratch PyTorch encoder-decoder LSTM with Bahdanau additive attention. Single-target training (`hi` → `en_anthropic`) on 14,591 line pairs.

```
python3 tier2_seq2seq_lstm.py
```

Step 14 — `tier2_seq2seq_lstm_v2.py`. Tier 2 v2: multi-target augmentation. Each `hi` line is paired with `en_anthropic`, `en_mistral`, and `en_t` as three separate training rows, tripling the effective training set to 43,673 pairs.

```
python3 tier2_seq2seq_lstm_v2.py
```

A.5 Tier 3 Scripts

Step 11 — `tier3_opus_mt.py`. Tier 3 v1: fine-tuning of Helsinki-NLP `opus-mt-hi-en` (MarianMT) using a two-phase schedule. Phase 1 on 50,000 IIT Bombay general sentence pairs (3 epochs, η=5×10^{-5}, batch 64); Phase 2 on poetry `hi` → `en_anthropic` (15 epochs, η=2×10^{-5}, batch 32).

```
python3 tier3_opus_mt.py
```

Step 15 — `tier3_opus_mt_v2.py`. Tier 3 v2: Phase 1 dropped, η=5×10^{-6}, weight decay 0.05, batch size 16, beam-search `no_repeat_ngram_size` 2, length penalty 0.8.

```
python3 tier3_opus_mt_v2.py
```

Step 17 — `tier3_opus_mt_v3a.py`. Tier 3 v3a: interleaved `en_anthropic` and `en_mistral` targets in alternating mini-batches.

```
python3 tier3_opus_mt_v3a.py
```

Step 18 — `tier3_opus_mt_v3b.py`. Tier 3 v3b: curriculum learning. Phase A on `en_anthropic`, then Phase B on `en_mistral` at η=2×10^{-6} for fine adjustment.

```
python3 tier3_opus_mt_v3b.py
```

A.6 Evaluation Scripts

Step 12 — `evaluate_all_tiers.py`. Evaluates the v1 set of three tiers on the 100 held-out test poems. Reports BLEU vs. Literal, BLEU vs. Style, Rhyme Density, and Syllable Alignment. Result table reproduced as Table 1.

```
python3 evaluate_all_tiers.py
```

Step 16 — `evaluate_all_tiers_v2.py`. Evaluates the v2 set of three tiers on the same 100 test poems. Result table reproduced as Table 2.

```
python3 evaluate_all_tiers_v2.py
```

Step 19 — `evaluate_all_tiers_final.py`. Final cross-version evaluation. Loads all eight saved checkpoints (T1 v1, T1 v2, T2 v1, T2 v2, T3 v1, T3 v2, T3 v3a, T3 v3b) and produces the unified comparison table reproduced as Table 3.

```
python3 evaluate_all_tiers_final.py
```

A.7 Demonstration

Step 20 — `streamlit_app.py`. Loads all eight local checkpoints plus three external API translators (Google Translate, Anthropic Claude, Mistral Large) and runs an input poem through 11 translators side-by-side, with per-model BLEU, rhyme density, and syllable alignment scores. Used as the live demo during the final presentation.

```
streamlit run streamlit_app.py
```